FLUTTER · DART · INTERVIEW PREP

Flutter

Tricky Interview Questions

48 questions that separate good Flutter developers from great ones. Clear answers, readable code and the traps interviewers love.

48	6	14+
Questions	Topics	Code snippets

main.dart

```dart
class Counter extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    setState(() {});  // Will this compile?
    return Text('Tricky?');
  }
}
```

Widgets · Container · Row & Column · Navigation · Lifecycle & Streams

Save it for your next interview · Share it with a Flutter developer

Contents

A focused tour of the Flutter concepts that interviewers love to probe. Each question ends with a clear answer, and the tricky parts are highlighted.

01	**Widgets & State** StatelessWidget, StatefulWidget and setState()	Q1–Q15	3
02	**The Container Widget** Sizing, constraints, padding, margin and transforms	Q16–Q25	6
03	**Row & Column Layouts** Axes, flex, overflow and unbounded constraints	Q26–Q33	8
04	**Navigation & Routing** Navigator, observers, named routes and arguments	Q34–Q37	10
05	**Lifecycle, Build & Streams** initState, mounted, rebuild triggers and stream types	Q38–Q45	12
06	**Bonus Round** Three new, harder questions added to this edition	Q46–Q48	16

How to read this guide

TRICK
The trap or misconception interviewers are hoping you fall into.

EXPLANATION
The reasoning behind the answer, so you can explain it in your own words.

GOOD TO KNOW
A short extra detail that makes your answer stand out.

NEW marks the extra questions added in this edition (Bonus Round).

01 Widgets & State

StatelessWidget, StatefulWidget and setState()

Q1–Q15 · 15 questions

01 Can a StatelessWidget be made Stateful?

TRICK

A common trap is to think a `StatelessWidget` can directly be made stateful. In reality, you need to convert it to a `StatefulWidget` to manage the state.

02 What happens if you call setState() in a StatelessWidget?

TRICK

A `StatelessWidget` has no `setState()` method, so trying to call it will result in a compilation error.

03 If a StatefulWidget doesn't change, why does it still get rebuilt?

EXPLANATION

`StatefulWidget` itself is immutable, but the state inside its associated `State` object changes, triggering a rebuild. It confuses many because even though the widget is technically "stateful," it behaves like a stateless component in terms of immutability.

04 When should you use a StatelessWidget vs. a StatefulWidget?

TRICK

Candidates often oversimplify. The key is to recognize the need for a `StatefulWidget` when the UI depends on user interaction or changes dynamically. `StatelessWidget` is suitable for UI that does not change after it's built.

05 Can you replace a StatefulWidget with a StatelessWidget if the widget's state is managed externally (e.g., via BLoC or Provider)?

ANSWER Yes, when state is managed externally using BLoC, Provider, or other state management solutions, you can use a `StatelessWidget` because it will rebuild automatically in response to external state changes. The trick is to understand that the widget can remain stateless even if the app is reactive.

06 Why does a StatefulWidget require two classes (Widget and State) instead of just one?

ANSWER Flutter separates the widget's structure (`StatefulWidget`) from its mutable state (`State`). This allows the widget to be immutable while still managing dynamic state within the `State` class. The dual-class structure ensures that the UI is rebuilt efficiently based on changes to the state.

07 Does setState() immediately rebuild the UI?

ANSWER No, calling `setState()` marks the widget as needing to rebuild in the next frame of Flutter's render cycle. This can confuse developers who expect the UI to update instantly. Instead, the framework optimizes rendering by scheduling a rebuild at the appropriate time.

08 What happens if you rebuild a StatelessWidget with the same configuration?

ANSWER When a `StatelessWidget` is rebuilt, the entire widget is reconstructed even if the configuration hasn't changed. This is different from StatefulWidgets, where only the state changes are applied, potentially leading to performance inefficiencies if overused.

> **GOOD TO KNOW**
> Declaring the widget with a `const` constructor lets Flutter skip rebuilding it when the configuration is identical.

09 How does a StatefulWidget maintain state when navigating between different screens?

ANSWER Normally, when a `StatefulWidget` is removed from the widget tree (e.g., navigating away), its state is destroyed. However, you can persist state across navigation using tools like `PageStorage`, `SharedPreferences`, or state management techniques like BLoC or Provider.

10 What would happen if you replaced a StatefulWidget's state with a different instance of the State object (e.g., manually)?

ANSWER Flutter ties each `State` object to its respective `StatefulWidget` instance. You cannot manually replace a `State` object. Flutter controls the lifecycle of state objects to ensure consistency, and replacing state manually would disrupt the framework's state management process.

11 Can you pass parameters to a StatefulWidget? If so, how?

ANSWER Yes, parameters can be passed to a `StatefulWidget` via its constructor, just like with StatelessWidgets. Inside the `State` class they are read through the `widget` property (for example `widget.title`). Care must be taken not to modify these parameters directly in `setState()` because that could trigger unexpected rebuilds and state inconsistencies.

12 What happens when the State object is destroyed in a StatefulWidget?

ANSWER When a `State` object is destroyed (e.g., navigating away), it is disposed of, and any resources tied to it, like animations or streams, need to be cleaned up in the `dispose()` method. Failing to clean up resources can lead to memory leaks and performance issues.

13 Can a StatelessWidget contain StatefulWidgets as its children?

ANSWER Yes, a `StatelessWidget` can contain StatefulWidgets. Even though the parent widget doesn't manage state, its children can handle their own state management. This allows for a mixture of stateless and stateful components in a widget tree.

14 Can you explain the lifecycle of a StatefulWidget?

ANSWER The lifecycle of a `StatefulWidget` consists of several key methods:

1. `createState()`: Creates the mutable state for this widget.
2. `initState()`: Called when this object is inserted into the tree; used for initialization.
3. `didChangeDependencies()`: Called when dependencies change.
4. `build()`: Describes how to display this widget in terms of other lower-level widgets.
5. `didUpdateWidget()`: Called when the widget configuration changes.
6. `setState()`: Notifies that the state has changed and triggers a rebuild.
7. `deactivate()`: Called when the widget is removed from the tree temporarily.
8. `dispose()`: Called when this object is removed from the tree permanently; used for cleanup.

> **GOOD TO KNOW**
> On a configuration update the order is `didUpdateWidget()` first, then `build()`.

15 Accessing stateful widget methods using a GlobalKey. Is this a valid approach?

ANSWER Using a `GlobalKey` to access methods or state in a `StatefulWidget` is technically valid but generally discouraged in production applications.

A `GlobalKey` allows access to a widget's associated state, enabling you to invoke methods or interact with its data. By assigning a `GlobalKey` to a widget, you can call methods from outside the widget tree, such as triggering animations, refreshing UI elements, or manipulating internal state.

Why it's discouraged:

1. **Tight Coupling:** Using `GlobalKey` tightly couples the caller to the internals of the widget, violating principles of encapsulation. This can make the widget harder to maintain and test.
2. **Performance Overhead:** `GlobalKey` comes with some overhead. Flutter must track and manage the widget's state more carefully, which can lead to performance issues in complex or large widget trees.
3. **Better Alternatives:** In most cases, there are better ways to manage communication between widgets, such as lifting state up, using callbacks, or employing state management solutions like Provider, BLoC, or Riverpod.

When to use:

- **Small Use Cases:** It's acceptable for small, isolated cases like accessing a child widget's scroll position or triggering animations, but it's best to avoid overuse for general state management.

02 The Container Widget

Sizing, constraints, padding, margin and transforms

Q16–Q25 · 10 questions

16 Does a Container always take up the full available space if no size is specified?

ANSWER No, a `Container` does not always take up the full available space unless constrained by its parent or given explicit width and height. By default, it only takes up as much space as its child needs. However, if no child is present and no size constraints are set, the `Container` might take up all available space depending on its parent widget.

17 What happens if you provide both a height and width to a Container, but the child has larger dimensions than the container's specified size?

ANSWER If both height and width are specified for a `Container` and the child widget's dimensions exceed those values, the child widget will be constrained to the container's size. This means the child will be clipped, resized, or rendered based on how its parent widget handles overflow, depending on other properties like `clipBehavior` or `FittedBox`.

18 Can you use both padding and margin in a Container widget? If so, how do they differ?

ANSWER Yes, you can use both `padding` and `margin` in a `Container` widget.

- **Padding** is the space between the container's child and its border. It adds space inside the container.
- **Margin** is the space outside the container, separating it from other widgets.

19 What happens if you set both width and constraints properties in a Container?

DART

```dart
Container(
  width: 200,
  constraints: BoxConstraints(maxWidth: 150),
)
```

ANSWER The `constraints` win. Flutter merges `width` into the constraints and then enforces the explicit `constraints`, so the container will have a maximum width of 150 pixels, regardless of the specified width of 200 pixels.

20 What will happen if you set a child widget that has its own constraints inside a Container with constraints?

```dart
Container(
  constraints: BoxConstraints(maxWidth: 100),
  child: ConstrainedBox(
    constraints: BoxConstraints(maxWidth: 200),
    child: Text('Hello'),
  ),
)
```

ANSWER The `Text` widget will be constrained to a maximum width of 100 pixels.

EXPLANATION

The outer `Container` defines a maximum width constraint of 100 pixels. Even though the inner `ConstrainedBox` allows for up to 200 pixels, it cannot exceed the constraints imposed by its parent `Container`.

21 Can you use a Container to apply a transformation (e.g., rotation) to its child?

ANSWER Yes, you can apply transformations like rotation or scaling using the `transform` property of a `Container`. This takes a `Matrix4` value, which allows you to transform the child widget. However, the transformation is applied to the entire container, including padding, margin, and decoration.

```dart
Container(
  transform: Matrix4.rotationZ(0.1),
  child: Text('Rotated Text'),
);
```

22 What is the relationship between child, decoration, and foregroundDecoration in a Container?

ANSWER The `child` is the content inside the container, `decoration` is applied behind the child (background), and `foregroundDecoration` is applied in front of the child (overlay). You can use `foregroundDecoration` for things like overlaying a semi-transparent color or image on top of the child while keeping the child visible underneath.

23 What is the impact of setting a Container's alignment versus aligning its child directly (e.g., using Align or Center)?

ANSWER Setting the Container's `alignment` aligns the child within the container's boundaries, which includes the padding and decoration. Using `Align` or `Center` inside the child widget aligns the child within its own constraints, which can sometimes behave differently, especially when working with containers that have specific sizes or constraints.

24 Is it possible for a Container to have a size of zero? How can this happen, and why would it be useful?

ANSWER Yes, a `Container` can have a size of zero if you provide constraints that force it to shrink (e.g., `BoxConstraints.tight(Size.zero)`). This can be useful in scenarios where you want to hide a widget temporarily while maintaining its position in the widget tree, such as when you're animating the visibility of a widget without removing it from the layout.

25 Can a Container have an infinite size? What scenarios can lead to this, and how can it be prevented?

ANSWER A `Container` can end up with infinite size if it's placed in a layout where the parent does not impose constraints (e.g., inside an unbounded `Column` or `ListView`). To prevent this, you must either provide explicit size constraints (using `width`, `height`, or `BoxConstraints`) or ensure the parent widget constrains the size.

03 Row & Column Layouts

Axes, flex, overflow and unbounded constraints

Q26–Q33 · 8 questions

26 What happens if you don't set a mainAxisSize for a Row or Column?

ANSWER By default, both `Row` and `Column` use `MainAxisSize.max`, which makes them take up as much space as possible in the main axis (horizontal for `Row` and vertical for `Column`). If you want them to take only as much space as their children need, you can set `mainAxisSize` to `MainAxisSize.min`.

27 What happens if a Row contains more widgets than the available horizontal space?

ANSWER If the total width of the children exceeds the available space, the `Row` will overflow. This results in a visual warning (a yellow and black striped bar) indicating the overflow, and an overflow error is logged in debug mode.

28 How does CrossAxisAlignment.stretch work in a Row compared to a Column?

ANSWER In a `Column`, `CrossAxisAlignment.stretch` makes all children expand horizontally to fill the width of the column. In a `Row`, it makes all children expand vertically to fill the height of the row. This is useful when you want widgets to take up all the available space along the cross axis.

29 What is the difference between using Expanded and Flexible in a Row or Column?

ANSWER `Expanded` forces a child to take up all the available space along the main axis, while `Flexible` allows the child to take up the available space but still respects its intrinsic size. If the child wants less space than what's available, `Flexible` allows that, whereas `Expanded` does not.

30 What happens if you nest a Column inside a Row or vice versa without setting any constraints?

ANSWER If you nest a `Column` inside a `Row` (or vice versa) without providing size constraints, the inner widget will take up as much space as its children need. However, if the parent (e.g., the `Row`) imposes constraints that don't allow the inner widget enough room, this can lead to layout issues like overflow or unintended clipping.

31 Can you add scrolling behavior to both Row and Column simultaneously? How would you achieve this?

ANSWER Not with a single widget, because a `Row` or `Column` does not scroll by itself. To scroll in both directions, nest two `SingleChildScrollView` widgets with different `scrollDirection` values (vertical outside, horizontal inside) and place the `Row` or `Column` in the inner one. This approach can introduce performance issues with large datasets, so use it with caution.

32 How does IntrinsicHeight and IntrinsicWidth interact with Row and Column?

ANSWER `IntrinsicHeight` makes a `Row` expand its height to match the tallest child, and `IntrinsicWidth` makes a `Column` expand its width to match the widest child. This can be performance-intensive, especially for complex layouts or lists, as it forces the layout to calculate the size of each child multiple times.

33 What happens when using a ListView inside a Column?

Using a `ListView` inside a `Column` can lead to layout issues, particularly because both `ListView` and `Column` are inherently unbounded in their height.

Issue: Unbounded Height Error. When you place a `ListView` directly inside a `Column`, the `Column` does not have a defined height, and the `ListView` tries to expand infinitely to fit its content. This can lead to a "RenderBox was given an unbounded height" error.

Solutions: To prevent this error, you need to give the `ListView` a bounded height.

- **Using Expanded:** Wrap the `ListView` with an `Expanded` widget. This allows the `ListView` to take up the remaining space in the `Column`.
- **Using a Fixed Height:** Give the `ListView` a specific height by wrapping it in a `Container` (or `SizedBox`) with a set height.
- **Using Flexible:** Similar to `Expanded`, `Flexible` allows you to specify how much space the `ListView` should take, with more control over the proportion of space taken.

```dart
Column(
  children: [
    Text('Header'),
    Flexible(
      flex: 1,
      child: ListView(
        children: [
          ListTile(title: Text('Item 1')),
          ListTile(title: Text('Item 2')),
          ListTile(title: Text('Item 3')),
        ],
      ),
    ),
  ],
)
```

04 Navigation & Routing

Navigator, observers, named routes and arguments

Q34–Q37 · 4 questions

34 What's the role of NavigatorObserver in Flutter?

ANSWER `NavigatorObserver` plays a crucial role in monitoring the navigation stack and managing route transitions within your application. It acts as a listener for route changes and can be useful for tracking navigation events for various purposes, such as analytics, logging, or custom behavior.

Key roles of NavigatorObserver

1. **Monitoring Navigation Events:** It observes changes in the navigation stack, such as when a new route is pushed, removed, or replaced. You can use this to track user behavior or app usage patterns.
2. **Customizing Navigation Behavior:** By extending `NavigatorObserver`, you can implement custom behavior during route changes, such as showing alerts, modifying transitions, or integrating with other services.
3. **Analytics and Logging:** You can log each route that a user visits, which can help in understanding user flows and improving user experience.
4. **Integration with State Management:** It can be used to trigger changes in your application's state based on navigation events, allowing for more dynamic user interfaces.

Common methods in NavigatorObserver

- `didPush(Route route, Route? previousRoute)`: Called when a new route has been pushed onto the navigator.
- `didPop(Route route, Route? previousRoute)`: Called when the current route has been popped off the navigator.
- `didRemove(Route route, Route? previousRoute)`: Called when a route has been removed from the navigator.
- `didReplace({Route? newRoute, Route? oldRoute})`: Called when a route has been replaced with a new route.

```dart
import 'package:flutter/material.dart';

class MyNavigatorObserver extends NavigatorObserver {
  @override
  void didPush(Route<dynamic> route, Route<dynamic>? previousRoute) {
    print('Pushed route: ${route.settings.name}');
  }

  @override
  void didPop(Route<dynamic> route, Route<dynamic>? previousRoute) {
    print('Popped route: ${route.settings.name}');
  }
}

// Register it on your app:
// MaterialApp(navigatorObservers: [MyNavigatorObserver()])
```

35 What is the difference between pushNamedAndRemoveUntil and pushReplacementNamed?

ANSWER `pushNamedAndRemoveUntil` adds a new route while removing all previous routes until a specified condition is met, effectively clearing the navigation stack. `pushReplacementNamed`, on the other hand, replaces the current route, maintaining the rest of the stack for potential back navigation.

36 How can you pass data between routes in Flutter?

ANSWER You can pass data when navigating to a new route using the `arguments` parameter in `Navigator.pushNamed`. The receiving route can access this data via `ModalRoute.of(context)!.settings.arguments`, allowing seamless data transfer between screens.

37 What is the purpose of the GlobalKey<NavigatorState>?

ANSWER The `GlobalKey<NavigatorState>` serves as a key that allows you to manage the state of a `Navigator` from anywhere in your application, enabling easier navigation control, especially when dealing with nested navigators or complex routing scenarios.

It provides a way to access the `NavigatorState` from outside the `BuildContext`, allowing you to perform navigation actions (like pushing and popping routes) without needing to pass around the context of the `Navigator`.

```dart
final GlobalKey<NavigatorState> navigatorKey =
    GlobalKey<NavigatorState>();

void main() {
  runApp(MyApp());
}

// Pass it to the app:
// MaterialApp(navigatorKey: navigatorKey, ...)

// Navigate from anywhere, with no BuildContext:
navigatorKey.currentState?.push(
  MaterialPageRoute(builder: (context) => SecondScreen()),
);
```

05 Lifecycle, Build & Streams

initState, mounted, rebuild triggers and stream types

Q38–Q45 · 8 questions

38 What's the difference between reassemble and build, and why is it important?

ANSWER `reassemble` is specifically for the hot reload feature in Flutter, allowing developers to update the UI without restarting the app. `build`, on the other hand, constructs the widget tree every time the widget needs to be redrawn. Understanding this distinction is crucial for effectively using hot reload, as certain states or side effects may need to be managed differently than during a normal build.

- **reassemble:** This method is primarily used in development. It is called when the application is hot-reloaded. You generally don't need to override this method unless you need to handle special cases when the app is reloaded.

39 Why might you want to delay certain initialization tasks beyond initState?

ANSWER Delaying initialization tasks beyond `initState` might be necessary for operations that depend on context, such as fetching data that involves inherited widgets or context-based services. Performing such tasks too early can lead to null exceptions or incomplete data, thus breaking the app's functionality.

40 When should you avoid using the async keyword in the initState method?

ANSWER `initState` cannot be marked `async` (it must stay synchronous and return `void`), and awaiting inside it is discouraged because it can lead to confusion regarding the widget lifecycle. The first `build` runs before your future completes, so the UI must handle a loading state. Instead, call a separate async method from `initState`, handle the loading state, and check `mounted` before calling `setState()` when the result arrives.

41 What is the potential drawback of using WidgetsBinding.instance.addPostFrameCallback in initState?

ANSWER The callback runs only after the first frame has been drawn, so the first build happens without your data. This means an extra rebuild (and possibly a visible flicker), and you must check `mounted` before touching state. Relying only on `initState` has its own risk: reading inherited widgets there (for example `Provider.of` with listening enabled) throws an error, because dependencies aren't fully set up yet; it's safer to perform such operations in `didChangeDependencies`.

DART / CAN FAIL

```dart
@override
void initState() {
  super.initState();
  // Assuming this value is dependent on an inherited widget.
  _data = Provider.of<MyDataProvider>(context).data; // Can cause an error.
}
```

Instead use this:

DART / SAFE

```dart
@override
void didChangeDependencies() {
  super.didChangeDependencies();
  _data = Provider.of<MyDataProvider>(context).data; // Safe to access here.
}
```

42 What is the role of mounted in the widget lifecycle?

ANSWER `mounted` is a boolean that indicates whether a widget is currently in the widget tree. It becomes `true` after `initState` and remains `true` until `dispose` is called.

TRICK

Calling `setState` on a widget that is not mounted (e.g., after an async call) leads to errors. Checking if a widget is `mounted` before calling `setState` can prevent this.

DART

```dart
Future<void> _load() async {
  final result = await repository.fetch();
  if (!mounted) return;          // widget may be gone by now
  setState(() => _items = result);
}
```

43 Why might you override build but not call super.build? What are the implications?

ANSWER Not calling `super.build` can lead to missing crucial functionalities provided by the parent class. This may result in the widget failing to render properly or losing its properties. Instead, always call `super.build(context)` if you're extending a custom widget that may have its own build logic.

GOOD TO KNOW
A classic case is `AutomaticKeepAliveClientMixin`: its `build` must be called via `super.build(context)` for the keep-alive behavior to work.

44 When does a widget rebuild happen?

ANSWER A widget rebuild occurs:

- When `setState` is called.
- When an inherited widget that the widget depends on changes.
- When a parent widget rebuilds.
- When the widget's configuration changes, leading to `didUpdateWidget`.

45 What are the key differences between Single-subscription & Broadcast Streams?

1. **Listeners.** Single Stream: only one listener can subscribe. If a second listener tries to access the stream, it will fail. Broadcast Stream: multiple listeners can subscribe simultaneously without any issues.
2. **Use Cases.** Single Stream: ideal for situations where only one part of the UI or application logic should react to events, such as a unique data source. Broadcast Stream: suitable for scenarios where multiple parts of the UI need to respond to the same events, such as notifications or state changes.
3. **Performance.** Single Stream: generally lighter on resources since it manages only one subscription. Broadcast Stream: slightly more overhead due to managing multiple listeners, but essential for use cases requiring multiple responses.

Single-subscription stream

DART

```dart
import 'dart:async';

void main() {
  final controller = StreamController<int>();

  // Using controller.sink.add
  controller.sink.add(1);

  // Using controller.add
  controller.add(2);

  controller.stream.listen((data) {
    print(data); // Output: 1, then 2 (events are buffered)
  });

  controller.close(); // Don't forget to close the controller
}
```

Broadcast stream

DART

```dart
import 'dart:async';

void main() {
  final controller = StreamController<int>.broadcast();

  // First listener
  controller.stream.listen((data) {
    print('Listener 1: $data');
  });

  // Second listener
  controller.stream.listen((data) {
    print('Listener 2: $data');
  });

  // Both listeners receive every event added after they subscribed
  controller.sink.add(1);
  controller.add(2);

  controller.close(); // Don't forget to close the controller
}
```

TRICK

A broadcast stream does not buffer events. Anything added before a listener subscribes is lost, so subscribe first and add data afterwards.

06 Bonus Round

Three new, harder questions added to this edition

Q46–Q48 · 3 questions

46 Why do Keys matter in a list of StatefulWidgets, and when would you pick ValueKey, ObjectKey, UniqueKey or GlobalKey? NEW

ANSWER When the tree is rebuilt, Flutter matches each old element with a new widget in the same slot if the `runtimeType` and `key` match. Without keys, reordering or removing items in a list of stateful widgets keeps each `State` attached to its position, so a checkbox, text input or animation can end up on the wrong item.

> **TRICK**
> Keys make state follow the data, not the position in the list.

- `ValueKey(item.id)`: the everyday choice when each item has a stable, unique id.
- `ObjectKey(item)`: when the identity of the whole object is the identity of the item.
- `UniqueKey()`: creates a brand-new key on every build, which throws the state away each time. Use it only when you want a reset.
- `GlobalKey`: unique across the app; lets state survive moving a widget to a new parent, but it is the most expensive option.

DART

```dart
ListView(
  children: [
    for (final todo in todos)
      TodoTile(key: ValueKey(todo.id), todo: todo),
  ],
)
```

47 Explain the three trees in Flutter: Widget, Element and RenderObject. Why is rebuilding widgets cheap? NEW

ANSWER Widgets are cheap, immutable descriptions of the UI. Elements are the long-lived instances that sit in the tree, hold the `State`, and connect a widget to its render object. RenderObjects do the expensive work of layout and painting.

- **Widget tree:** short-lived configuration, recreated on every rebuild.
- **Element tree:** the real tree, reused between rebuilds. `BuildContext` is actually the `Element`.
- **RenderObject tree:** performs layout, hit testing and painting.

On a rebuild, each element asks whether the new widget can take over its slot. If it can, the element and render object are simply updated with the new configuration. If not, that whole subtree is thrown away and rebuilt.

DART

```dart
// Simplified from the Flutter framework
static bool canUpdate(Widget oldWidget, Widget newWidget) {
  return oldWidget.runtimeType == newWidget.runtimeType &&
      oldWidget.key == newWidget.key;
}
```

TRICK

`setState()` doesn't rebuild the `State` object. It marks its Element as dirty so the framework calls `build()` again on the next frame.

48 Why can using BuildContext after an await be a bug, and how do you fix it? NEW

ANSWER While your code is suspended at `await`, the user may have navigated away and the widget may be gone. Using its `context` afterwards can throw errors such as looking up a deactivated widget's ancestor, and calling `setState()` after `dispose()` fails too. The Dart lint `use_build_context_synchronously` flags this.

The fix: check `mounted` (inside a `State`) or `context.mounted` (Flutter 3.7+, works anywhere) after every async gap, before touching the context. You can also capture things like `Navigator.of(context)` before the `await`.

DART

```dart
Future<void> _submit() async {
  final navigator = Navigator.of(context); // capture before the gap
  await repository.save(form);

  if (!mounted) return;                    // guard after the gap
  ScaffoldMessenger.of(context)
      .showSnackBar(const SnackBar(content: Text('Saved')));
  navigator.pop();
}
```

Quick recap

Five ideas to carry into your next interview

1. Widgets are immutable configurations; **State** and Elements are what live on between rebuilds.
2. **setState()** schedules a rebuild for the next frame; it never rebuilds instantly.
3. Constraints go down, sizes go up, and the parent sets the position.
4. Always clean up in **dispose()**, and check **mounted** after every await.
5. Use **keys** whenever stateful items can be reordered, inserted or removed.

Found this useful?

Save this PDF, repost it to help other Flutter developers, and share which question tripped you up the most.